Hyunkio and **claude** TIE Fighter 챌린지: 크리퍼 그리기, my_utils.py 분리 318f8ef · 2 minutes ago History

In [1]:
```
# my_utils.py를 같은 폴더에 두고 실행
import matplotlib.pyplot as plt
from my_utils import INF, draw_creeper
```

In [2]:
```
fig, ax = plt.subplots(figsize=(7,7))

ax.set_xlim(-1.1*INF, 1.1*INF)
ax.set_ylim(-1.1*INF, 1.1*INF)
ax.set_aspect('equal')
ax.axis('off')

draw_creeper(ax)

plt.show()
```



Hyunkio and claude TIE Fighter 챌린지: 크리퍼 그리기, my_utils.py 분리 318f8ef · 3 minutes ago History

Code Blame 259 lines (216 loc) · 5.64 KB Raw

```python
import numpy as np
from matplotlib.patches import Polygon

INF = 1000        # 그림판 크기
LEN = 100         # 처음 사각형의 크기


# 동차좌표로 바꿔 행렬을 곱하고 2차원으로 되돌림: q = m p
def transform(p, m):
    hp = np.array([p[0], p[1], 1.0])

    q = np.dot(m, hp)

    q = np.array([q[0], q[1]])
    return q


# 이동: T(dx, dy)
def translate(p, dx, dy):
    m = np.array([[1.0, 0.0, dx],
                  [0.0, 1.0, dy],
                  [0.0, 0.0, 1.0]])
    return transform(p, m)


# 회전: R(theta)
def rotate(p, theta):
    c = np.cos(theta)
    s = np.sin(theta)
    m = np.array([[c,  -s,  0.0],
                  [s,   c,  0.0],
                  [0.0, 0.0, 1.0]])
    return transform(p, m)


# 크기: S(sx, sy)
def scale(p, sx, sy):
    m = np.array([[sx,  0.0, 0.0],
                  [0.0, sy,  0.0],
                  [0.0, 0.0, 1.0]])
    return transform(p, m)


# y축 방향 기울기: Shy(sy)
def shear_y(p, sy):
    m = np.array([[1.0, 0.0, 0.0],
                  [sy,  1.0, 0.0],
                  [0.0, 0.0, 1.0]])
    return transform(p, m)


# y축 반전: Rfy
def reflect_y(p):
    m = np.array([[-1.0, 0.0, 0.0],
                  [ 0.0, 1.0, 0.0],
                  [ 0.0, 0.0, 1.0]])
    return transform(p, m)


# 사각형의 모든 꼭짓점에 같은 변환 적용
def transform_rect(rect, f, *args):
    return [f(p, *args) for p in rect]


# 크리퍼 색상표
PALETTE = [
    '#010601', '#143b11', '#264c22', '#0e7b0c',
    '#33612d', '#398731', '#486944', '#4e9144',
    '#646c62', '#5d9557', '#5cba4f', '#729e6c',
    '#79ca6c', '#8c9c8a', '#9bd192', '#c5dfc0',
]

# 면의 무늬: 글자 하나가 칸 하나, 값은 PALETTE 번호(16진수)
```

```
HEAD_LEFT = [
    "b35bd994",
    "4557477d",
    "512b612d",
    "51075019",
    "97b11d47",
    "b620017d",
    "d6000074",
    "791bd147",
]
HEAD_RIGHT = [
    "44611244",
    "42811648",
    "28621228",
    "48448888",
    "28648646",
    "48268448",
    "28622488",
    "46668482",
]
HEAD_TOP = [
    "ae5aaa57",
    "aea7aa5c",
    "33ef77ce",
    "3a9facae",
    "fffbabce",
    "999fff77",
    "f9fcccff",
    "99baac7c",
]

BODY_LEFT = [
    "176b24",
    "267534",
    "722553",
    "762233",
    "6b5933",
    "944954",
    "6dbb57",
    "647b95",
    "922b94",
    "4566b6",
    "95b297",
    "95b197",
]
BODY_RIGHT = [
    "b94624",
    "244464",
    "3b6264",
    "526222",
    "546262",
    "692111",
    "692426",
    "642464",
    "b24624",
    "941211",
    "452446",
    "456422",
]

FRONT_FOOT_LEFT = [
    "336d334d",
    "336d336d",
    "3d773b77",
    "96779677",
    "10101010",
    "01010101",
]
FRONT_FOOT_RIGHT = [
    "4266",
    "2461",
    "8621",
    "8441",
    "8446",
    "1626",
]
FRONT_FOOT_TOP = [
    "f8f9",
    "cc7e",
    "aace",
    "eeee",
    "f9f9",
    "cc7e",
    "aace",
    "eeec",
]

```

```
BACK_FOOT_LEFT = [
    "336d331d",
    "336d626d",
    "3d7b6b77",
    "96779677",
    "10101010",
    "01010101",
]
BACK_FOOT_RIGHT = [
    "4266",
    "2461",
    "8621",
    "8441",
    "8446",
    "1626",
]
BACK_FOOT_TOP = [
    "f8f9",
    "cc7e",
    "aace",
    "eeee",
    "f9f9",
    "cc5e",
    "29ce",
    "eeec",
]


CELL_W = 50        # 칸의 가로 폭
SLOPE = 0.5        # 입체로 보이게 하는 기울기


# 색을 지정해 사각형 그리기 (칸 사이 틈이 보이지 않게 테두리도 같은 색)
def draw_color_rectangle(ax, p1, p2, p3, p4, color):
    my_polygon = Polygon([p1, p2, p3, p4], facecolor=color, edgecolor=color, linewidth=0.5)
    ax.add_patch(my_polygon)


# 오른쪽 면의 칸: 크기 → 이동 → y축 방향 기울기
def right_cell(rect, i, j, cell_h):
    q = transform_rect(rect, scale, CELL_W/LEN, cell_h/LEN)
    q = transform_rect(q, translate, i*CELL_W, -(j+1)*cell_h)
    q = transform_rect(q, shear_y, SLOPE)
    return q


# 왼쪽 면의 칸: 오른쪽 면의 칸을 y축 반전
def left_cell(rect, i, j, cell_h):
    q = right_cell(rect, i, j, cell_h)
    q = transform_rect(q, reflect_y)
    return q


# 윗면의 칸: 이동 → 45도 회전 → 세로로 눌러 마름모로 만들기
def top_cell(rect, i, j):
    s = np.sqrt(2)*CELL_W/LEN
    q = transform_rect(rect, translate, i*LEN, j*LEN)
    q = transform_rect(q, rotate, np.pi/4)
    q = transform_rect(q, scale, s, s*SLOPE)
    return q


# 칸을 상자의 앞쪽 모서리 위치로 옮겨서 그리기
def place_cell(ax, q, front, code):
    q = transform_rect(q, translate, front[0], front[1])
    draw_color_rectangle(ax, *q, PALETTE[int(code, 16)])


# 상자 하나 그리기: front는 앞쪽 모서리의 위 꼭짓점
def draw_box(ax, rect, front, cell_h, left, right, top=None):
    n = len(left[0])

    for j, row in enumerate(left):
        for i, code in enumerate(row):
            place_cell(ax, left_cell(rect, n-1-i, j, cell_h), front, code)

    for j, row in enumerate(right):
        for i, code in enumerate(row):
            place_cell(ax, right_cell(rect, i, j, cell_h), front, code)

    if top:
        for j, row in enumerate(top):
            for i, code in enumerate(row):
                place_cell(ax, top_cell(rect, i, n-1-j), front, code)


def draw_creeper(ax):
```

```
        # 처음 사각형의 네 꼭짓점
        p1 = np.array([0, 0])
        p2 = np.array([LEN, 0])
        p3 = np.array([LEN, LEN])
        p4 = np.array([0, LEN])
        rect = [p1, p2, p3, p4]

        # 뒤에 있는 부품부터 그리기: 뒷발 → 몸통 → 앞발 → 머리
        draw_box(ax, rect, (300, -461), 58.7, BACK_FOOT_LEFT, BACK_FOOT_RIGHT, BACK_FOOT_TOP)
        draw_box(ax, rect, (-4, 157), 57.7, BODY_LEFT, BODY_RIGHT)
        draw_box(ax, rect, (-101, -651), 60.5, FRONT_FOOT_LEFT, FRONT_FOOT_RIGHT, FRONT_FOOT_TOP)
        draw_box(ax, rect, (0, 621), 60.5, HEAD_LEFT, HEAD_RIGHT, HEAD_TOP)
```